

SNAGGA

Projekt-Dokumentation & Betriebshandbuch

Stand: Juni 2026 | Version 1.0

Dieses Dokument erklärt das gesamte Snagga-Projekt — für Einsteiger verständlich und gleichzeitig technisch vollständig. Es deckt Architektur, alle beteiligten Dienste, automatische Prozesse, Wartungsaufgaben, Sicherheitsmassnahmen und Notfallprozeduren ab.

Domain	snagga.de
Affiliate	Amazon snagga-21
Status	Live & produktionsreif
Kosten/Mo	~1.40 EUR (nur Domain)

1. Was ist Snagga? (Einfach erklärt)

Snagga ist eine Website (snagga.de), die automatisch die besten Amazon-Deals für Deutschland, Österreich und die Schweiz sucht und übersichtlich anzeigt. Jeder Deal hat eine Bewertung (0–100), die zeigt wie gut der aktuelle Preis im Vergleich zum historischen Durchschnitt ist.

Wie verdient Snagga Geld? Wenn jemand über einen Snagga-Link etwas bei Amazon kauft, bekommt der Betreiber eine kleine Provision (ca. 1–10% je nach Kategorie). Für den Käufer kostet das nichts extra.

Der Weg eines Deals — von Amazon zu dir

Schritt	Was passiert	Wer macht es	Wie oft
1	CamelCamelCamel veröffentlicht Preisänderungen im RSS-Feed	Camel RSS-Feed	Laufend
2	Snagga Backend liest den RSS-Feed aus	Render Server (Python)	Täglich 03:00
3	Deals werden bewertet und in der Datenbank gespeichert	Snagga Algorithmus	Täglich 03:00
4	Nutzer öffnet snagga.de im Browser	Nutzer	Jederzeit
5	Frontend fragt Backend nach aktuellen Deals	React App (Vercel)	Bei Seitenöffnung
6	Backend liest Deals aus Datenbank und gibt sie aus	FastAPI (Render)	Bei Anfrage
7	Deals werden schön angezeigt, Nutzer klickt auf Deal	Browser des Nutzers	Jederzeit
8	Nutzer landet auf Amazon mit Affiliate-Tag in der URL	Amazon	Bei Klick
9	Nutzer kauft → Snagga bekommt Provision	Amazon	Bei Kauf

2. Alle beteiligten Dienste & Anbieter

Snagga nutzt insgesamt 9 externe Dienste. Alle ausser der Domain sind kostenlos. Jeder Dienst hat eine klar definierte Aufgabe.

Dienst	Anbieter	Aufgabe	Kosten	Login
Frontend (React App)	Vercel vercel.com	Zeigt die Website an. Hostet die React-App als statische Dateien.	Gratis	GitHub (auto)
Backend (API Server)	Render render.com	Beantwortet Anfragen der App. Holt Deals, liest/schreibt DB.	Gratis (Free Tier)	render.com
Datenbank (PostgreSQL)	Render render.com	Speichert alle Produkte und Preishistorie dauerhaft.	Gratis (90 Tage!)	render.com
Domain	Hostinger hpanel.hostinger.com	snagga.de gehört uns. DNS-Einstellungen hier verwalten.	~1.40 EUR /Monat	hpanel. hostinger.com
Code- Verwaltung	GitHub github.com	Speichert den gesamten Quellcode. Jeder Push deployt automatisch.	Gratis	github.com/ bluesleeismee
Überwachung	UptimeRobot uptimerobot.com	Pingt Server alle 5 Min. Hält Render wach. Alarm bei Ausfall.	Gratis	uptimerobot.com
E-Mail Weiterleitung	ImprovMX improvmx.com	contact@snagga.de leitet weiter an davidpauli@gmx.ch	Gratis	app.improvmx.com

Preisdaten	CamelCamelCamel camelcamelcamel.com	Gratis RSS-Feed mit amazon.de Preis- änderungen.	Gratis (kein Login)	—
Affiliate Programm	Amazon PartnerNet	Provisionen bei Käufen über unsere Links.	Gratis (snagga-21)	affiliate- program. amazon.de

■■ WICHTIG: Die PostgreSQL-Datenbank auf Render läuft nach 90 Tagen ab (ca. 20. September 2026). Danach muss eine neue erstellt und die DATABASE_URL in Render Environment aktualisiert werden.

3. Technische Architektur

Frontend (Was der Nutzer sieht)

Das Frontend ist eine React Progressive Web App (PWA), gebaut mit Vite. Es wird auf Vercel gehostet und ist unter snagga.de erreichbar. Die App ist mobile-first gestaltet (max. 480px Breite). Jedes Mal wenn Code auf GitHub gepusht wird, deployt Vercel automatisch die neue Version.

Datei	Inhalt
src/App.jsx	Hauptkomponente, Tab-Navigation, globaler State (Theme, Watchlist, Alerts)
src/pages/DealsPage.jsx	Deal-Feed mit Suche, Kategoriefilter, Sortierung
src/pages/WatchlistPage.jsx	Gespeicherte Deals (lokal im Browser)
src/pages/AlertsPage.jsx	Preis-Alerts (lokal im Browser, noch keine E-Mails)
src/pages/SettingsPage.jsx	Theme, Kategorien, manueller Refresh, Impressum-Link
src/pages/LegalPage.jsx	Impressum, Datenschutz, Affiliate-Hinweis
src/components/DealCard.jsx	Einzelne Deal-Karte mit Preishistorie-Chart
src/api.js	Verbindung zum Backend (BASE URL aus VITE_API_URL)
src/index.css	CSS-Variablen (Dark/Light Theme), globale Stile

Backend (Der unsichtbare Server)

Das Backend ist ein Python FastAPI Server, der auf Render läuft. Er hat 4 Endpunkte die das Frontend aufruft, und einen automatischen Job der täglich Deals holt.

Endpunkt	Methode	Was er macht
/deals	GET	Gibt Liste aller Deals zurück (mit Filter, Sortierung, Suche)
/product/{asin}	GET	Gibt ein einzelnes Produkt mit voller Preishistorie zurück
/categories	GET	Gibt alle verfügbaren Kategorien zurück
/refresh	POST	Holt manuell neue Deals (max. alle 5 Minuten)

/health	GET+HEAD	Statuscheck — UptimeRobot pingt hier alle 5 Minuten
---------	----------	---

Datei	Inhalt
main.py	FastAPI App, alle Endpunkte, CORS-Konfiguration, Startup-Logik
database.py	PostgreSQL-Verbindung via asyncpg, Tabellen-Erstellung
scraper.py	RSS-Parser, Deal-Bewertungsalgorithmus, Datenbankschreiber
scheduler.py	APScheduler — startet täglichen Job um 03:00 Uhr
requirements.txt	Alle Python-Abhängigkeiten (fastapi, asyncpg, feedparser, ...)
.env (lokal)	Lokale Umgebungsvariablen (wird nicht nach GitHub gepusht)

Datenbank (PostgreSQL)

Die Datenbank hat zwei Tabellen:

Tabelle	Felder (wichtigste)	Inhalt
products	asin, name, brand, category, current_price, original_price, deal_score, affiliate_url	Ein Eintrag pro Amazon-Produkt. Wird bei jedem Update überschrieben.
price_history	asin, price, timestamp	Preispunkte pro Produkt und Datum. Für den Mini-Chart in der App. Simuliert für neue Produkte.

Umgebungsvariablen (Render Environment)

Diese Variablen müssen auf Render gesetzt sein damit das Backend läuft:

Variable	Wert (Beispiel)	Wozu
DATABASE_URL	postgresql://snagga_db_user:...@...	Verbindung zur PostgreSQL-Datenbank
FRONTEND_URL	https://snagga.de	Erlaubte CORS-Origin (Sicherheit)
SCHEDULER_HOUR	3	Stunde des täglichen Updates (03:00)
SCHEDULER_MINUTE	0	Minute des täglichen Updates

4. Was läuft automatisch?

Snagga ist so gebaut, dass es sich weitgehend selbst betreibt. Folgende Prozesse laufen ohne manuelles Eingreifen:

Prozess	Wann	Was passiert	Zuverlässigkeit
---------	------	--------------	-----------------

UptimeRobot Ping	Alle 5 Min.	Ruft /health auf. Hält Render-Server wach. Sendet E-Mail bei Ausfall.	★★★★★ Sehr zuverlässig
Deal-Update (APScheduler)	Täglich 03:00 Uhr	Liest CCC RSS-Feeds aus, bewertet Deals schreibt in PostgreSQL.	★★★★■ Nicht 100% sicher bei Render-Neustarts
Vercel Auto-Deploy	Bei jedem git push	Frontend wird automatisch neu gebaut und deployed. Dauert ~1 Minute.	★★★★★ Sehr zuverlässig
Render Auto-Deploy	Bei jedem git push	Backend wird automatisch neu gestartet mit dem neuen Code.	★★★★■ Zuverlässig
Fallback Seed-Daten	Wenn RSS nicht erreichbar	Statt RSS-Daten werden 10 vorbereitete Testprodukte angezeigt.	★★★★★ Immer verfügbar

Deployment-Ablauf bei Code-Änderung

1. Änderungen werden lokal auf D:\Claude\DealRadar gemacht → 2. git add / git commit / git push → 3. GitHub empfängt den Code → 4a. Vercel baut Frontend neu (ca. 1 Min) → 4b. Render deploys Backend neu (ca. 2-3 Min) → 5. Beide Seiten live mit neuer Version.

5. Sicherheitsmassnahmen

Massnahme	Was es schützt	Wie implementiert
CORS-Einschränkung	Nur snagga.de darf das Backend ansprechen. Kein fremder Code kann Daten abrufen.	allowed_origins=['https://snagga.de', 'https://www.snagga.de']
Rate Limiting /refresh	Verhindert Missbrauch des Refresh-Buttons. Max. 1 Aufruf alle 5 Minuten.	RateLimitMemoryCooldownTimer in main.py
Parametrisierte SQL	SQL-Injection ist nicht möglich. Benutzereingaben werden nie direkt in SQL eingebaut.	asyncpg \$1, \$2 Platzhalter in allen Queries
Kein Wildcard CORS	Früherer Fehler: '*' erlaubte alle Webseiten. Jetzt explizit eingeschränkt.	Behoben am 22.06.2026
HTTPS überall	Alle Verbindungen sind verschlüsselt. Keine unverschlüsselten HTTP-Anfragen.	Vercel + Render erzwingen HTTPS
Gitignore	.env, snagga.db, __pycache__ werden nie auf GitHub hochgeladen.	.gitignore Datei im Projektordner

6. Bekannte Risiken & offene Punkte

Priorität	Risiko	Wann relevant	Lösung
-----------	--------	---------------	--------

■ Kritisch	PostgreSQL läuft ab (Render Free = 90 Tage)	Ca. 20. Sept. 2026	Neue DB erstellen auf Render, DATABASE_URL aktualisieren, oder auf \$7/Mo upgraden.
■ Mittel	N+1 Query Problem (50 Deals = 51 DB-Abfragen)	Bei >100 gleichzeitigen Nutzern spürbar	Preishistorie in einem einzigen JOIN laden.
■ Mittel	Kein API-Caching (Jede Anfrage fragt DB ab)	Bei mehr Traffic	/deals Antworten 5 Min cachen (z.B. mit Redis).
■ Mittel	APScheduler unzuverlässig (03:00 Job könnte ausfallen)	Bei Render-Neustart nachts	Render Cron Job als Backup einrichten.
■ Mittel	Amazon Bilder laden sometimes nicht (404)	Zufällig	Fallback-Bild wenn Amazon-URL nicht lädt.
■ Klein	localStorage mit altem 'dr_' Prefix	Nur bei Redesign relevant	Keys umbenennen auf 'sn_' beim Redesign.
■ Klein	Kein Error-Monitoring (Fehler nur in Render-Logs)	Wenn etwas kaputt geht	Sentry Free Tier einbinden.
■ Geplant	Volle Desktop-Ansicht fehlt (max 480px)	Beim Redesign	Responsive Layout für Tablet und Desktop.
■ Geplant	Amazon PA API (bessere Datenquelle)	Nach 3 Verkäufen	PA API ersetzt CCC RSS, echte Preise und Bilder.
■ Geplant	E-Mail Alerts (Preis-Benachrichtigungen)	Spätere Phase	Mailgun oder Resend + Login-System.

7. Wartungsaufgaben & Kalender

Diese Aufgaben müssen manuell erledigt werden:

Wann	Aufgabe	Wo	Dauer
20. Sept. 2026 (PFLICHT)	Neue PostgreSQL-Datenbank erstellen auf Render, DATABASE_URL aktualisieren, Render neu deployen.	render.com	~15 Min.
01. Juli 2027	Domain snagga.de verlängern (automatische Verlängerung aktiviert, aber prüfen!)	hpanel.hostinger.com	~5 Min.
Monatlich	Render Logs kurz prüfen ob Deal-Updates täglich funktionieren. UptimeRobot Status prüfen.	render.com Logs uptimerobot.com	~5 Min.
Bei Bedarf	Wenn snagga.de nicht lädt: Render Logs checken, manuellen Deploy starten falls nötig.	render.com	~10 Min.
Nach 3 Verkäufen	Amazon PA API beantragen und in scraper.py einbinden.	affiliate-program.amazon.de + scraper.py	~2 Stunden

8. Notfall-Checkliste

Problem: snagga.de lädt nicht

Schritt	Was prüfen	Wo
1	Ist die Domain aktiv?	hpanel.hostinger.com → Domains → snagga.de
2	Ist das Frontend deployed?	vercel.com → snagga → Deployments
3	Zeigen DNS-Einstellungen auf Vercel?	hpanel.hostinger.com → DNS/Nameserver
4	Funktioniert snagga.onrender.com direkt?	Browser: https://snagga.onrender.com/health

Problem: Keine Deals werden angezeigt

Schritt	Was prüfen	Wo
1	Backend läuft?	https://snagga.onrender.com/health → muss '{status: ok}' zeigen
2	Datenbank hat Daten?	health-Endpoint zeigt 'products: 0' → manuelles Refresh nötig
3	Manuelles Refresh	snagga.de → Einstellungen → 'Deals jetzt aktualisieren'
4	Render Logs prüfen	render.com → snagga → Logs → Fehlermeldungen suchen
5	PostgreSQL abgelaufen?	Render → snagga-db → Status (Available / Expired)

Problem: UptimeRobot meldet Server down

Schritt	Was prüfen	Wo
1	Ist es ein echter Ausfall?	Browser: https://snagga.onrender.com/health direkt aufrufen
2	Render Server schläft?	Erster Aufruf dauert ~20 Sek. → normal, dann wieder ok
3	Render crasht in Loop?	render.com → Logs → Fehlermeldung → Code-Fix nötig
4	Neues Deployment starten	render.com → snagga → Manual Deploy → Deploy latest commit

9. Schnell-Referenz: Alle wichtigen URLs & Zugänge

Beschreibung	URL / Info
Live Website	https://snagga.de
Backend API	https://snagga.onrender.com
Health Check	https://snagga.onrender.com/health

GitHub Repo	https://github.com/bluesleeismee/snagga
Vercel Dashboard	https://vercel.com/dashboard
Render Dashboard	https://render.com → My project → snagga
Hostinger Domain	https://hpanel.hostinger.com → Domains → snagga.de
UptimeRobot	https://uptimerobot.com → Monitoring
ImprovMX E-Mail	https://app.improvmx.com → snagga.de
Amazon Affiliate	https://affiliate-program.amazon.de (Tag: snagga-21)
Kontakt E-Mail	contact@snagga.de → leitet weiter an davidpauli@gmx.ch
Lokaler Code	D:\Claude\DealRadar
Git Push Befehl	<code>cd /d D:\Claude\DealRadar → git add . → git commit -m '...' → git pus</code>